

To what degree do you believe the following statement: Nearly anybody enrolled at Berkeley could achieve an A in CS61B.

- Assume they can spend the entire summer preparing beforehand, hire a tutor during the semester, etc.
- a. Strongly disagree
- b. Disagree
- c. Neutral
- d. Agree
- e. Strongly agree

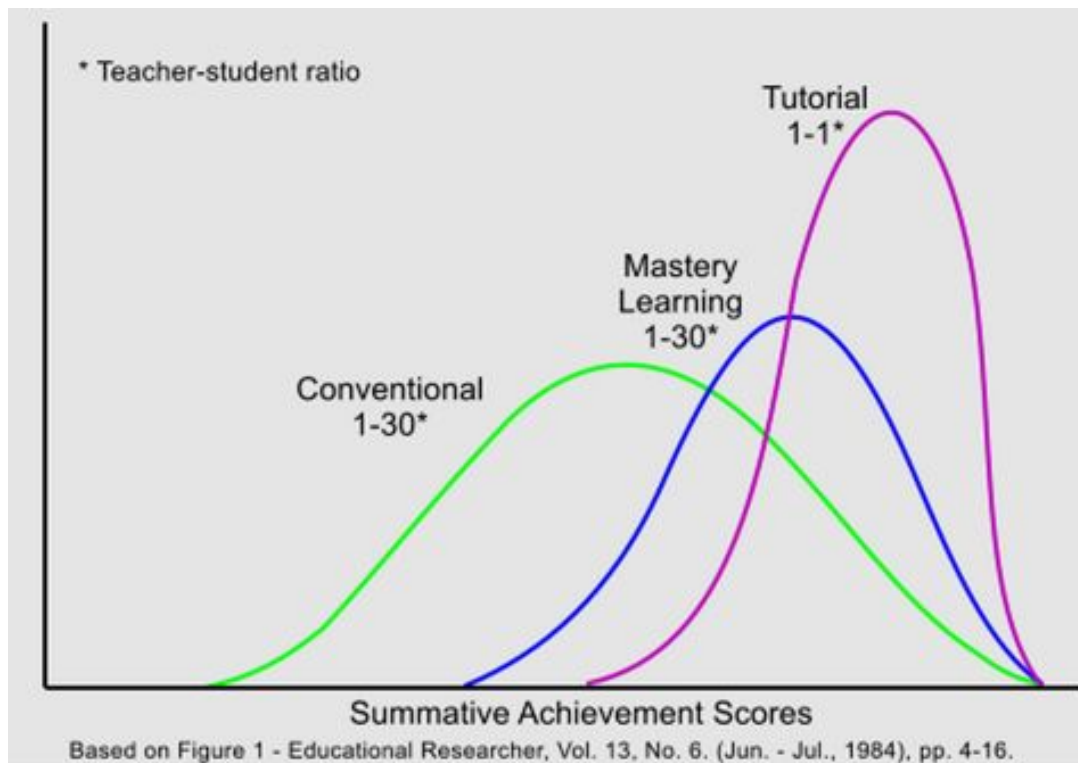


Students can be thought of as having either a “growth” mindset or a “fixed” mindset (based on research by Carol Dweck).

- “In a **fixed mindset** students believe their basic abilities, their intelligence, their talents, are just fixed traits. They have a certain amount and that's that, and then their goal becomes to look smart all the time and never look dumb.”
 - Perhaps most damningly, having to put in effort is a sign of weakness!
- “In a **growth mindset** students understand that their talents and abilities can be developed through effort, good teaching and persistence. They don't necessarily think everyone's the same or anyone can be Einstein, but they believe everyone can get smarter if they work at it.”

Growing up and even through undergrad I was very much part of the “fixed mindset” camp.

Bloom's Two Sigma Problem: In one experiment, student randomly picked for 1-on-1 teaching performed similarly to the top 2% of a simultaneous lecture course.



In Sp16, I gave students the option to fail intentionally.

- On average, students were 2.5 letter grades higher the second time, e.g. someone in the middle of the B range got an A the second time.

Possible interpretation: There exists some sort of preparation for 61B that helps students do much better.

Quick note: This midterm was more challenging for you than I expected!

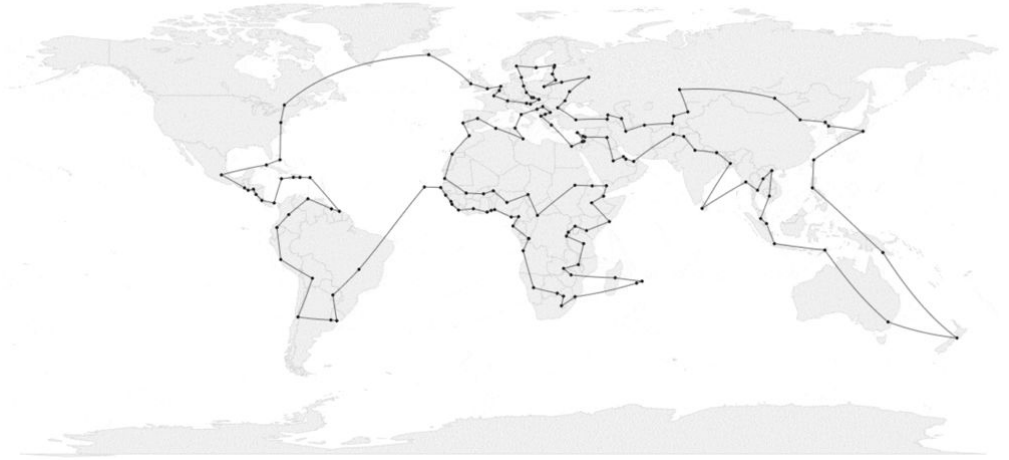
- Example, the `resize` function was directly from the project.
 - Mean score on this problem was ~52%.
 - My best guess was that it would be closer to 75%.
- The `ejectND` problem was very similar to problem from Spring 2018 midterm.
 - Spring 2018 solve rate for blank 3 was ~85-95%.
 - Fall 2025 solve rate for blank 3 was ~55-65%.

Still analyzing!

- Students came in with less preparation?
- Students working in isolation and getting low quality assistance from LLMs?
- Change in cheat sheet policy?

Any questions or thoughts?

Distance: 80,652 miles
Temperature: 2
Iterations: 1,000,000



Lecture 14

Asymptotics 2

CS61B, Fall 2025 @ UC Berkeley

Josh Hug and Josh Hug

Given a code snippet, we can express its runtime as a function $R(N)$, where N is some property of the input of the function (often the size of the input).

Rather than finding $R(N)$ exactly, we instead usually only care about the order of growth of $R(N)$.

One approach (not universal):

- Choose a representative operation, and let $C(N)$ be the count of how many times that operation occurs as a function of N .
- Determine order of growth $f(N)$ for $C(N)$, i.e. $C(N) \in \Theta(f(N))$
 - Often (but not always) we consider the worst case count.
- If operation takes constant time, then $R(N) \in \Theta(f(N))$

Nested For Loops

Lecture 14, CS61B, Fall 2025

Asymptotic Analysis Examples

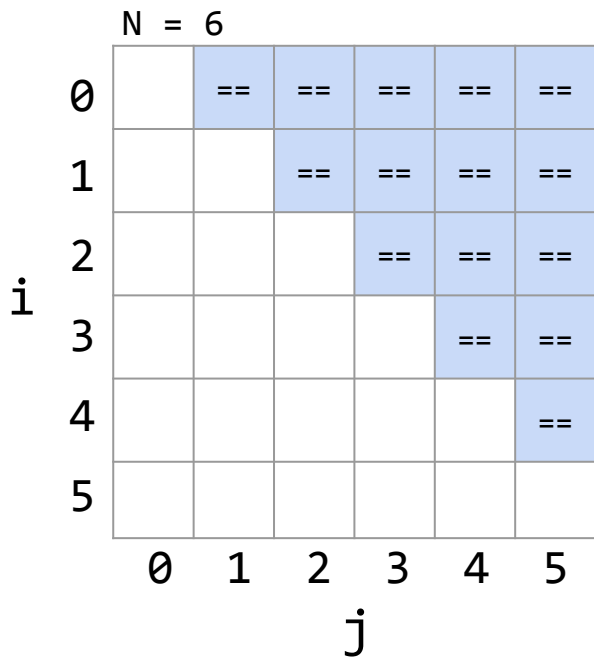
- **Nested For Loops**
- A Trickier Nested For Loop
- There is No Magic Shortcut for Asymptotic Analysis
- Recursive Function Analysis

More Asymptotic Notation

- Big O
- Big Theta

Loops Example 1: Based on Exact Count

Find the order of growth of the worst case runtime of dup1.



```
int N = A.length;
for (int i = 0; i < N; i += 1)
    for (int j = i + 1; j < N; j += 1)
        if (A[i] == A[j])
            return true;
return false;
```

Worst case number of == operations:

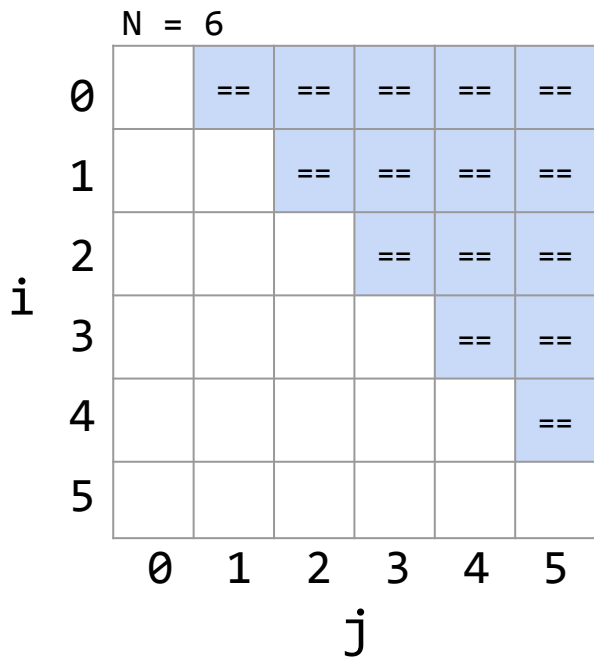
$$C(N) = 1 + 2 + 3 + \dots + (N - 2) + (N - 1) = N(N-1)/2$$

operation	worst case count
==	$\Theta(N^2)$

Worst case runtime: $\Theta(N^2)$

Loops Example 1: Simpler Geometric Argument

Find the order of growth of the worst case runtime of dup1.



```
int N = A.length;
for (int i = 0; i < N; i += 1)
    for (int j = i + 1; j < N; j += 1)
        if (A[i] == A[j])
            return true;
return false;
```

Worst case number of == operations:

- Given by area of right triangle of side length N-1.
- Area is $\Theta(N^2)$.

operation	worst case count
==	$\Theta(N^2)$

Worst case runtime: $\Theta(N^2)$

Let $R(N)$ be the runtime of `scan`. Find a simple $f(N)$ such that $R(N) \in \Theta(f(N))$. By simple, we mean there should be no unnecessary multiplicative constants or additive terms.



```
public static void scan(int N) {  
    int s = 0;  
    for (int i = 2; i <= N; i += 1) {  
        for (int j = i-1; j <= i+1; j += 1) {  
            s += i*j;  
        }  
    }  
}
```

- A. 1
- B. $\log N$
- C. N
- D. $N \log N$
- E. N^2
- F. Other

Note that there's only one case for this code and thus there's no distinction between "worst case" and otherwise.

Loops Example 2: Arithmetic Argument

Find the order of growth of the worst case runtime of scan.

```
public static void scan(int N) {  
    int s = 0;  
    for (int i = 2; i <= N; i += 1) {  
        for (int j = i-1; j <= i+1; j += 1) {  
            s += i*j;  
        }  
    }  
}
```

Total number of $s += i*j$ operations:

- $i = 2$: 3 operations
- $i = 3$: 3 operations
- ...
- $i = N$: 3 operations

$$\begin{aligned} C(N) &= \overbrace{3 + 3 + 3 + \dots + 3}^{N-1} \\ &= 3(N-1) \\ &\in \Theta(N) \end{aligned}$$

Loops Example 2: Arithmetic Argument

Find the order of growth of the worst case runtime of scan.

```
public static void scan(int N) {  
    int s = 0;  
    for (int i = 2; i <= N; i += 1) {  
        for (int j = i-1; j <= i+1; j += 1) {  
            s += i*j;  
        }  
    }  
}
```

$C(N)$ is the count of $s += i*j$.

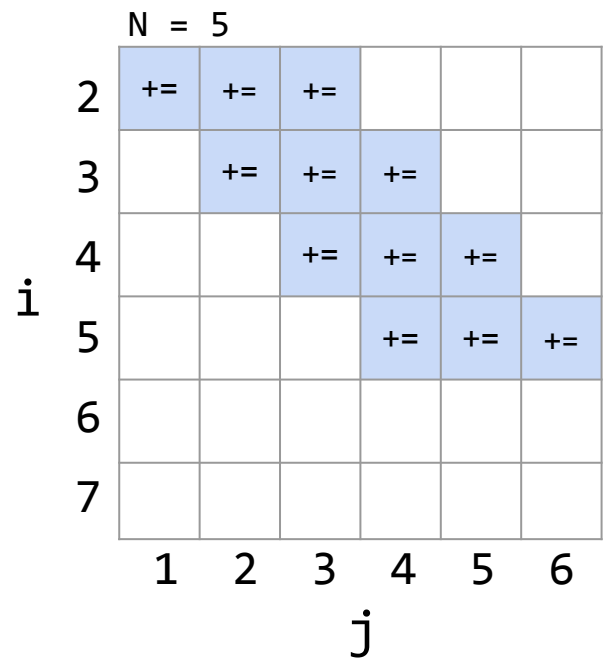
- Since $C(N) \in \Theta(N)$, and $C(N)$ is a good cost model, that means $R(N) \in \Theta(N)$.



Example of a bad cost model, counting $i = 2$ calls.

Loops Example 2: Geometric Argument

Can also use geometry to find order of growth of the worst case runtime of scan.



```
public static void scan(int N) {
  int s = 0;
  for (int i = 2; i <= N; i += 1) {
    for (int j = i-1; j <= i+1; j += 1) {
      s += i*j;
    }
  }
}
```

Worst case number of $s += i*j$ operations:

- Given by area of line of width 3 and length $N - 1$.
- Overall area is $\Theta(N)$

operation	Count C(N)
$s += i*j$	$\Theta(N)$

Runtime: $\Theta(N)$

A Trickier Nested For Loop

Lecture 14, CS61B, Fall 2025

Asymptotic Analysis Examples

- Nested For Loops
- **A Trickier Nested For Loop**
- There is No Magic Shortcut for Asymptotic Analysis
- Recursive Function Analysis

More Asymptotic Notation

- Big O
- Big Theta

SP26 ToDo: Start by asking them to give the number of print calls when $N = 4$.

Find a simple $f(N)$ such that the runtime $R(N) \in \Theta(f(N))$. By simple, we mean there should be no unnecessary multiplicative constants or additive terms.

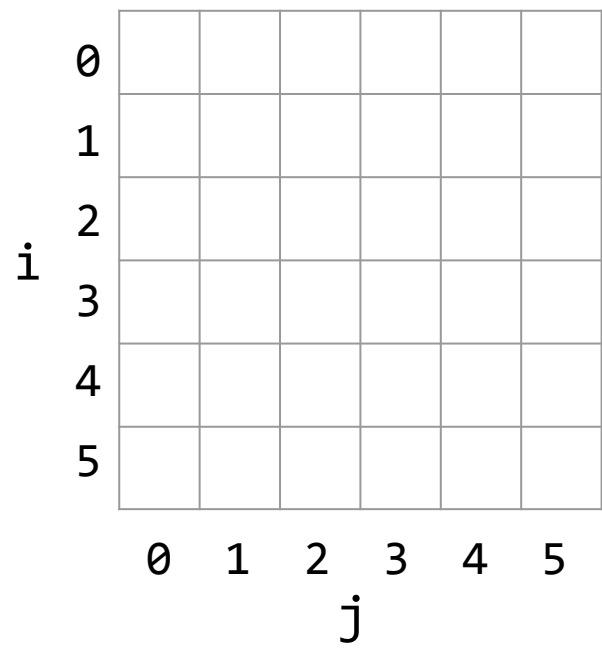
```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```



- | | |
|-------------|---------------|
| A. 1 | D. $N \log N$ |
| B. $\log N$ | E. N^2 |
| C. N | F. Other |

Note that there's only one case for this code and thus there's no distinction between "worst case" and otherwise.

Loops Example 3: Prelude to Attempt #2



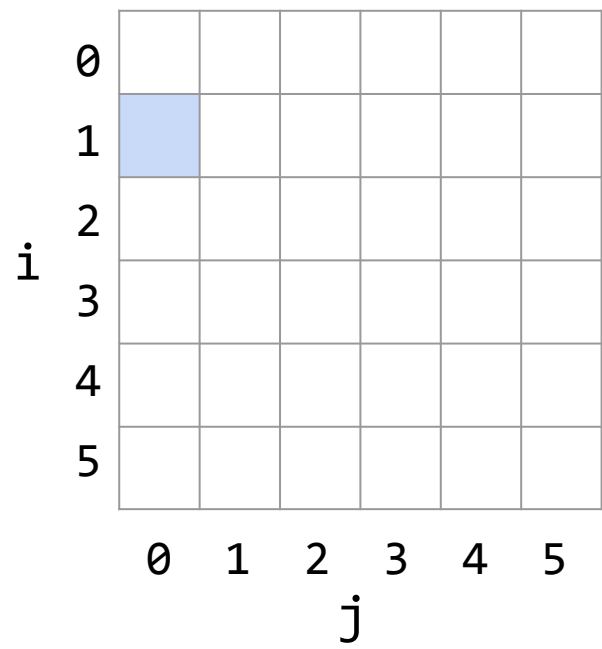
```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```

Find a simple $f(N)$ such that the runtime $R(N) \in \Theta(f(N))$.

Cost model $C(N)$, println("hello") calls:

N	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
C(N)	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?

Loops Example 3: Prelude to Attempt #2



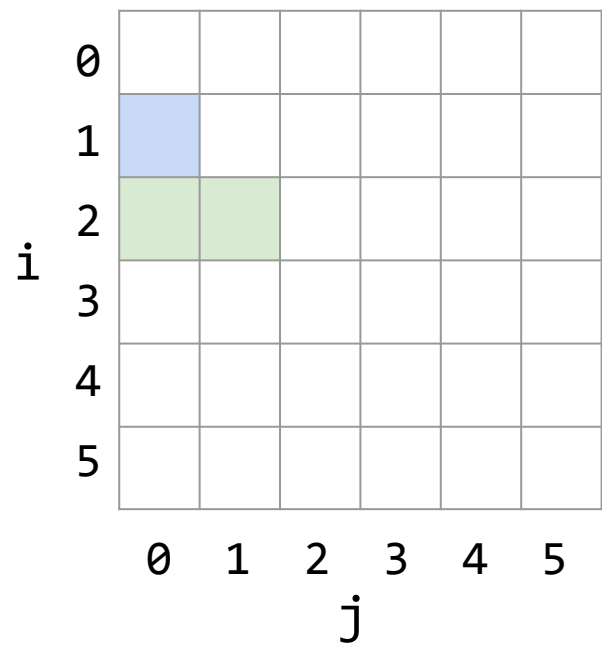
```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```

Find a simple $f(N)$ such that the runtime $R(N) \in \Theta(f(N))$.

Cost model $C(N)$, println("hello") calls:

N	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
C(N)	1																	

Loops Example 3: Prelude to Attempt #2



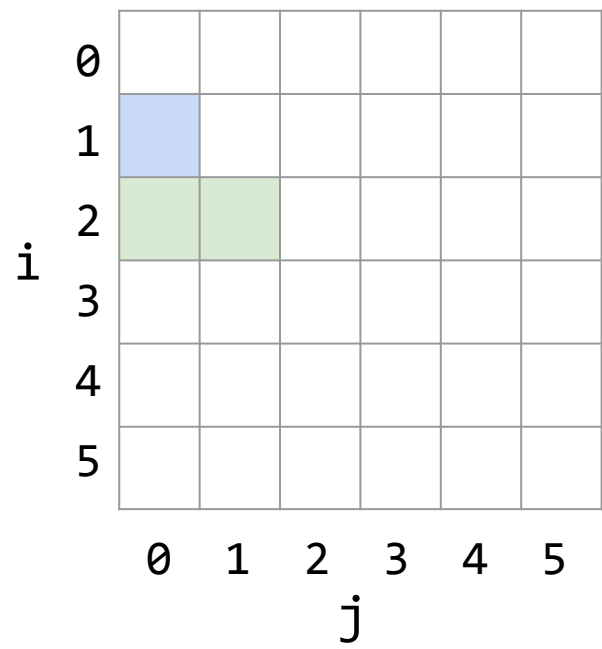
```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```

Find a simple $f(N)$ such that the runtime $R(N) \in \Theta(f(N))$.

Cost model $C(N)$, println("hello") calls:

N	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
C(N)	1	3																

Loops Example 3: Prelude to Attempt #2



```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```

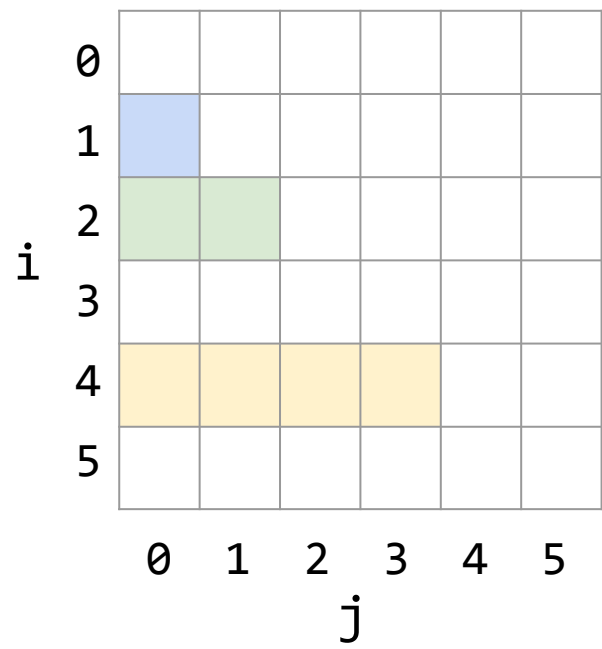
Find a simple $f(N)$ such that the runtime $R(N) \in \Theta(f(N))$.

Cost model $C(N)$, println("hello") calls:

N	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
C(N)	1	3	3															

N=2 and 3 both print 3 times

Loops Example 3: Prelude to Attempt #2



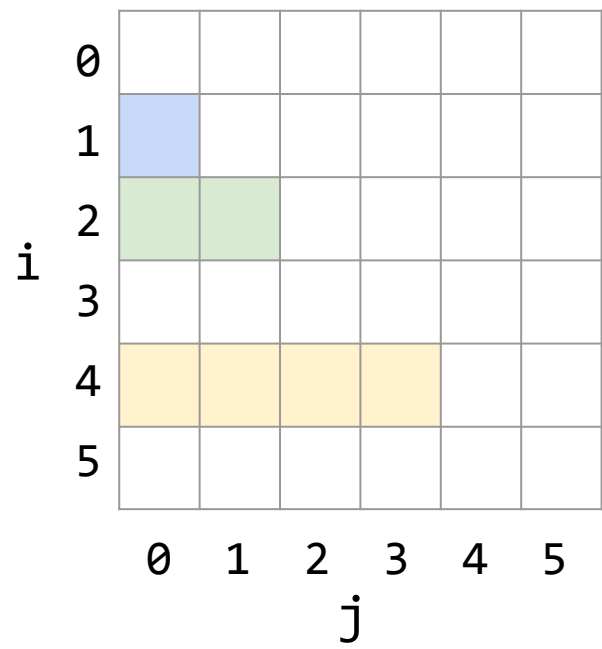
```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```

Find a simple $f(N)$ such that the runtime $R(N) \in \Theta(f(N))$.

Cost model $C(N)$, println("hello") calls:

N	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
C(N)	1	3	3	7														

Loops Example 3: Prelude to Attempt #2



```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```

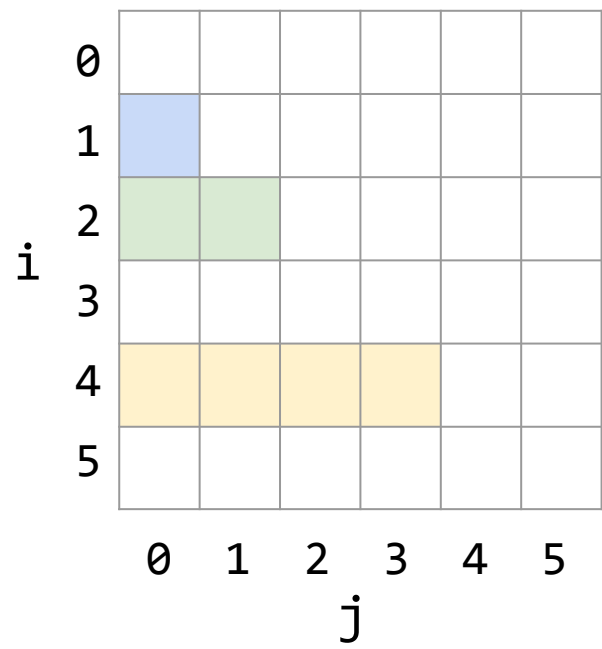
Find a simple $f(N)$ such that the runtime $R(N) \in \Theta(f(N))$.

Cost model $C(N)$, println("hello") calls:

N	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
C(N)	1	3	3	7	7	7	7											

N=4,5,6,7 all print 7 times

Loops Example 3: Prelude to Attempt #2



```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```

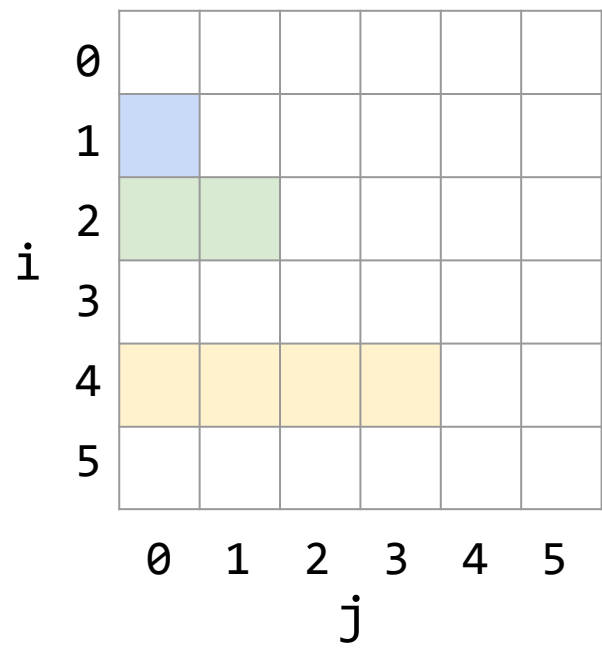
Find a simple $f(N)$ such that the runtime $R(N) \in \Theta(f(N))$.

Cost model $C(N)$, println("hello") calls:

N	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
C(N)	1	3	3	7	7	7	7	15	15	15	15	15	15	15	15			

These N all print 15 times

Loops Example 3: Prelude to Attempt #2



```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```

Find a simple $f(N)$ such that the runtime $R(N) \in \Theta(f(N))$.

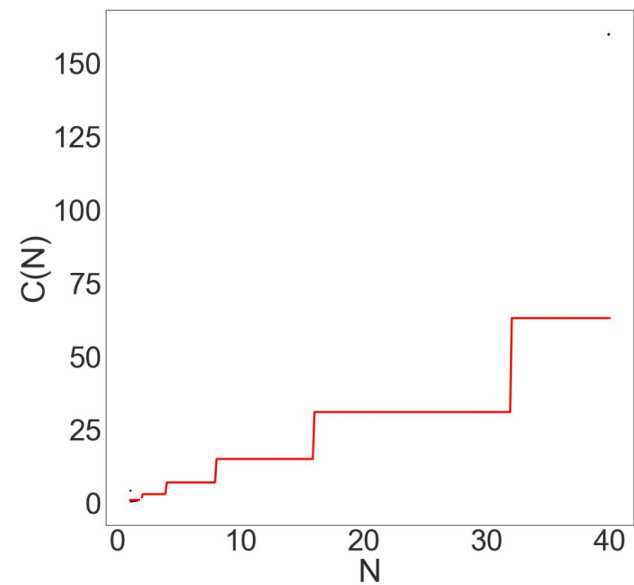
Cost model $C(N)$, `println("hello")` calls:

N	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
C(N)	1	3	3	7	7	7	7	15	15	15	15	15	15	15	15	31	31	31

$C(N) = 1 + 2 + 4 + \dots + N$, if N is a power of 2

Loops Example 3: Prelude to Attempt #2

We're trying to find the order of growth of $C(N)$.



Cost model $C(N)$, `println("hello")` calls:

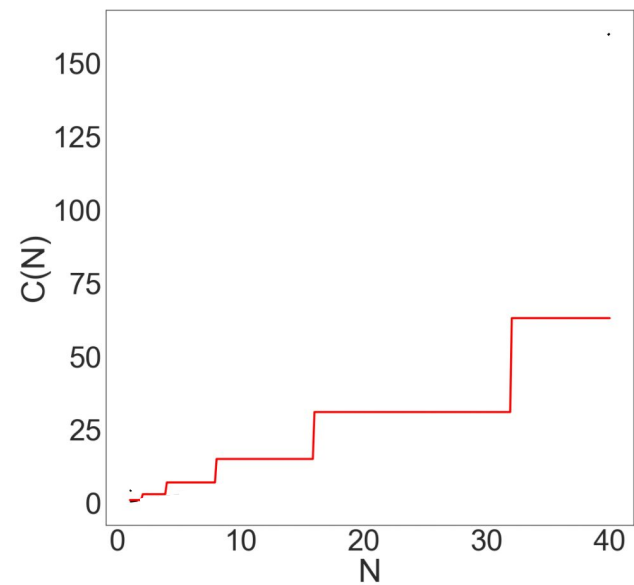
N	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
C(N)	1	3	3	7	7	7	7	15	15	15	15	15	15	15	15	31	31	31

$C(N) = 1 + 2 + 4 + \dots + N$, if N is a power of 2

Loops Example 3: Prelude to Attempt #2

We're trying to find the order of growth of $C(N)$.

- We know that if N is a power of 2,
 - $C(N) = 1 + 2 + 4 + \dots + N$
- Example: $N = 8$
 - $C(N) = 1 + 2 + 4 + 8 = 15$
- Example: $N = 16$
 - $C(N) = 1 + 2 + 4 + 8 + 16 = 31$



Cost model $C(N)$, `println("hello")` calls:

N	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
C(N)	1	3	3	7	7	7	7	15	15	15	15	15	15	15	15	31	31	31

$C(N) = 1 + 2 + 4 + \dots + N$, if N is a power of 2

Find a simple $f(N)$ such that the runtime $R(N) \in \Theta(f(N))$.

- A. 1

B. $\log N$

C. N
- D. $N \log N$

E. N^2

F. Other



```
public static void printParty(int N) {
    for (int i = 1; i <= N; i = i * 2) {
        for (int j = 0; j < i; j += 1) {
            System.out.println("hello");
        }
        int ZUG = 1 + 1;
    }
}
```

Cost model $C(N)$, `println("hello")` calls:

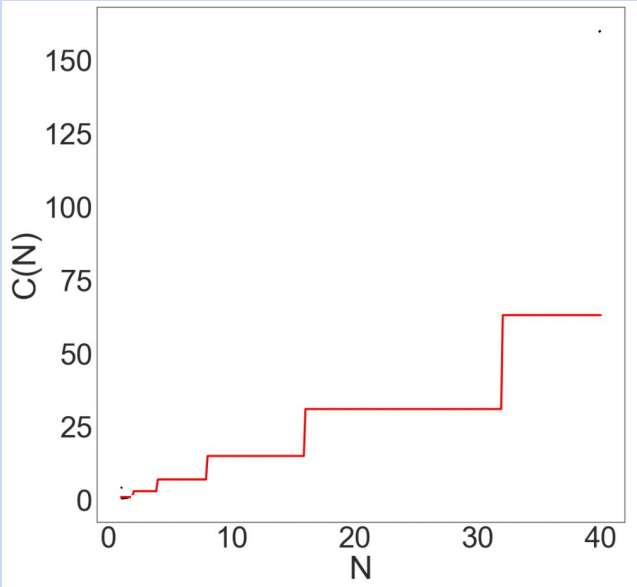
N	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
C(N)	1	3	3	7	7	7	7	15	15	15	15	15	15	15	15	31	31	31

$C(N) = 1 + 2 + 4 + \dots + N$, if N is a power of 2

Loops Example 3: Prelude to Attempt #2

We're trying to find the order of growth of $C(N)$.

- We know that if N is a power of 2,
 - $C(N) = 1 + 2 + 4 + \dots + N$
- Example: $N = 8$
 - $C(N) = 1 + 2 + 4 + 8 = 15$
- Example: $N = 16$
 - $C(N) = 1 + 2 + 4 + 8 + 16 = 31$
- Try to find a simple formula.
 - What is $C(N)$ if N is a power of 2?



Cost model $C(N)$, `println("hello")` calls:

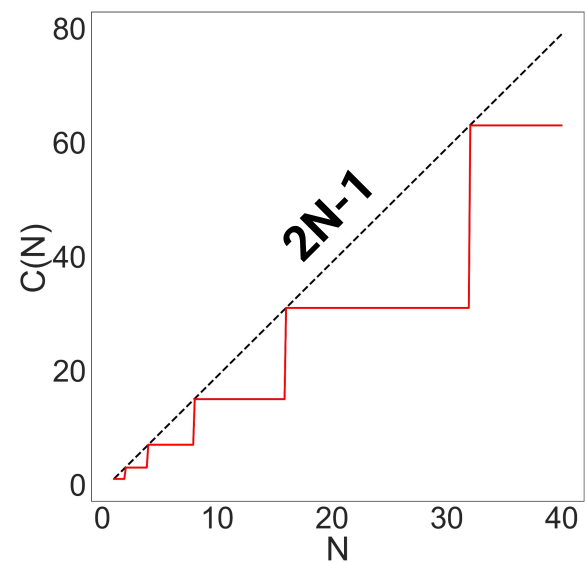
N	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
C(N)	1	3	3	7	7	7	7	15	15	15	15	15	15	15	15	31	31	31

$C(N) = 1 + 2 + 4 + \dots + N$, if N is a power of 2

Loops Example 3: Prelude to Attempt #3

We're trying to find the order of growth of $C(N)$.

- We know that if N is a power of 2,
 - $C(N) = 1 + 2 + 4 + \dots + N$
- Example: $N = 16$
 - $C(N) = 1 + 2 + 4 + 8 + 16 = 31$
- Give a general formula if N is a power of 2:
 - $C(N) = 2N - 1$
 - (See next two skipped slides for proof)



Cost model $C(N)$, `println("hello")` calls:

N	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
C(N)	1	3	3	7	7	7	7	15	15	15	15	15	15	15	15	31	31	31

$C(N) = 1 + 2 + 4 + \dots + N$, if N is a power of 2

Proof that $1 + 2 + 4 + \dots + N = 2N - 1$

Claim: If N is a power of 2, then $1 + 2 + 4 + \dots + N = 2N - 1$

Proof by induction:

- Base case, let $N = 1$.
 - $2 \cdot 1 - 1 = 1$

Inductive hypothesis: Sum of powers of 2 up to $N = 1 + 2 + 4 + \dots + N = 2N - 1$

- After N , the next power of two is $2N$.
- Thus, we want to show $1 + 2 + 4 + \dots + N + 2N = 4N - 1$
- We start by assuming the inductive hypothesis: $1 + 2 + 4 + \dots + N = 2N - 1$
 - Then we add $2N$ to both sides.
 - This gives us that $1 + 2 + 4 + \dots + N + 2N = 2N - 1 + 2N$.

$$= 4N - 1$$

You might ask: Why are you writing this so weirdly? You wrote:

- Suppose N is a power of 2, then $1 + 2 + 4 + \dots + N = 2N - 1$.

But it seems more natural to write this as: Suppose N is a power of 2, i.e. $N = 2^k$ for some k , then:

$$1 + 2 + 2^2 + \dots + 2^k = \sum_{i=0}^k 2^i = 2^{k+1} - 1$$

Experience has shown me that 61B students get k and N confused and this more formal framing ends up being a distraction.

Find a simple $f(N)$ such that $C(N) \in \Theta(f(N))$.

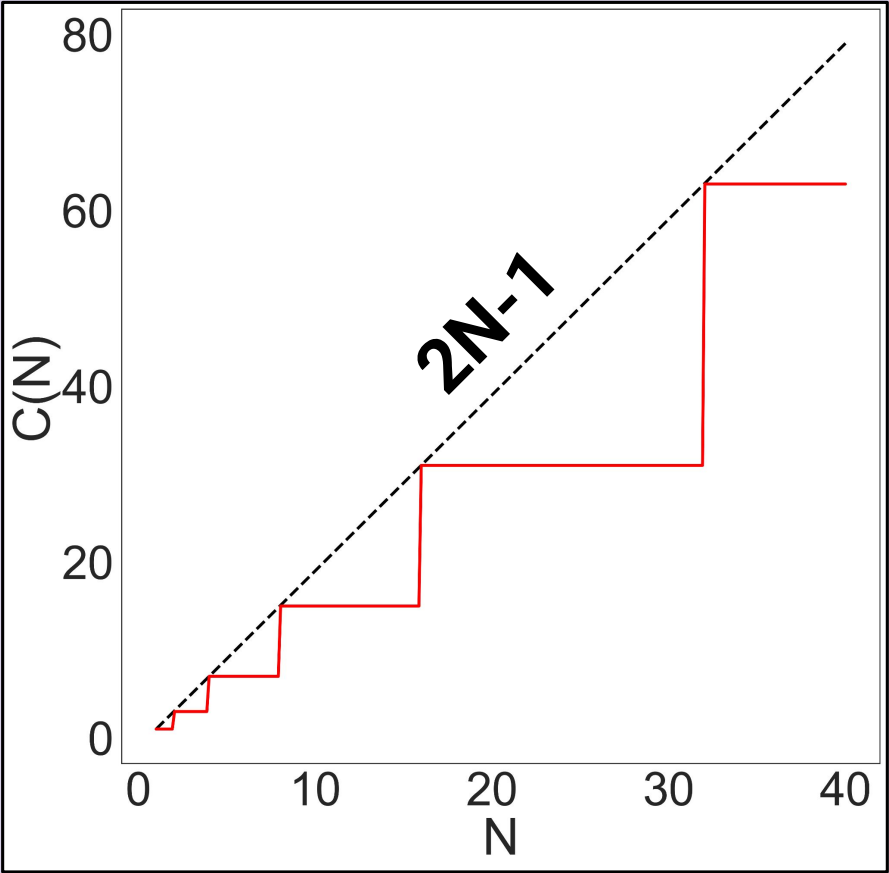
- A. 1

B. $\log N$

C. N
- D. $N \log N$

E. N^2

F. Other



Solution: $C(N)$ is Linear

Find a simple $f(N)$ such that $C(N) \in \Theta(f(N))$.

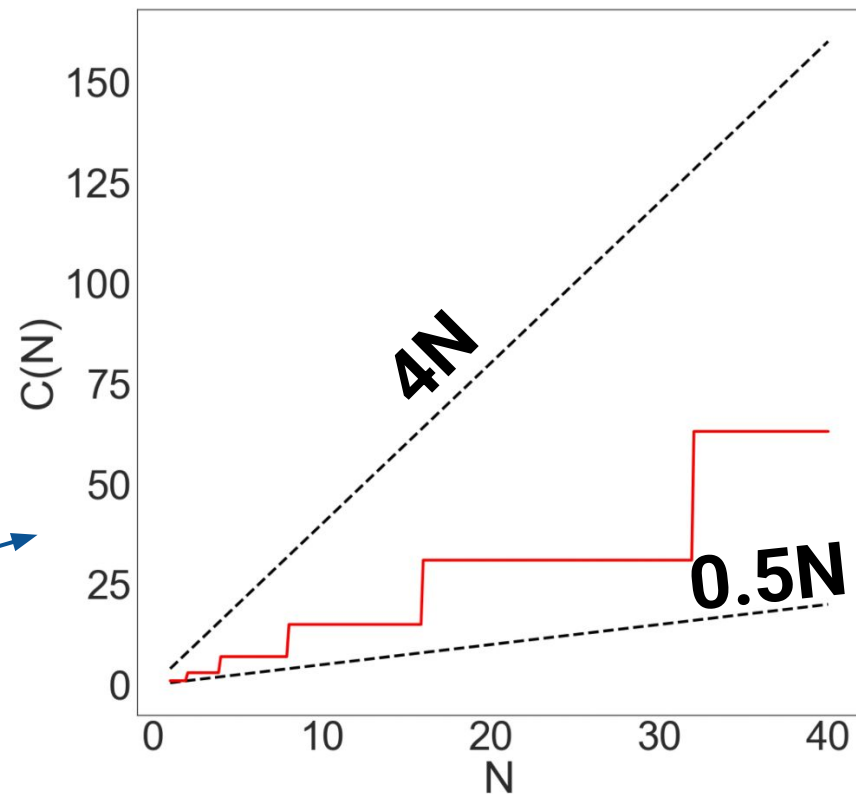
- | | |
|--------------------------|---------------|
| A. 1 | D. $N \log N$ |
| B. $\log N$ | E. N^2 |
| C. N | F. Other |

Recall that $C(N) \in \Theta(N)$ means that we can find k_1 and k_2 such that for large N :

- $k_1 * N \leq C(N) \leq k_2 * N$

For example:

- $k_1 = 0.5, k_2 = 4$
- $k_1 = 0.001, k_2 = 103$



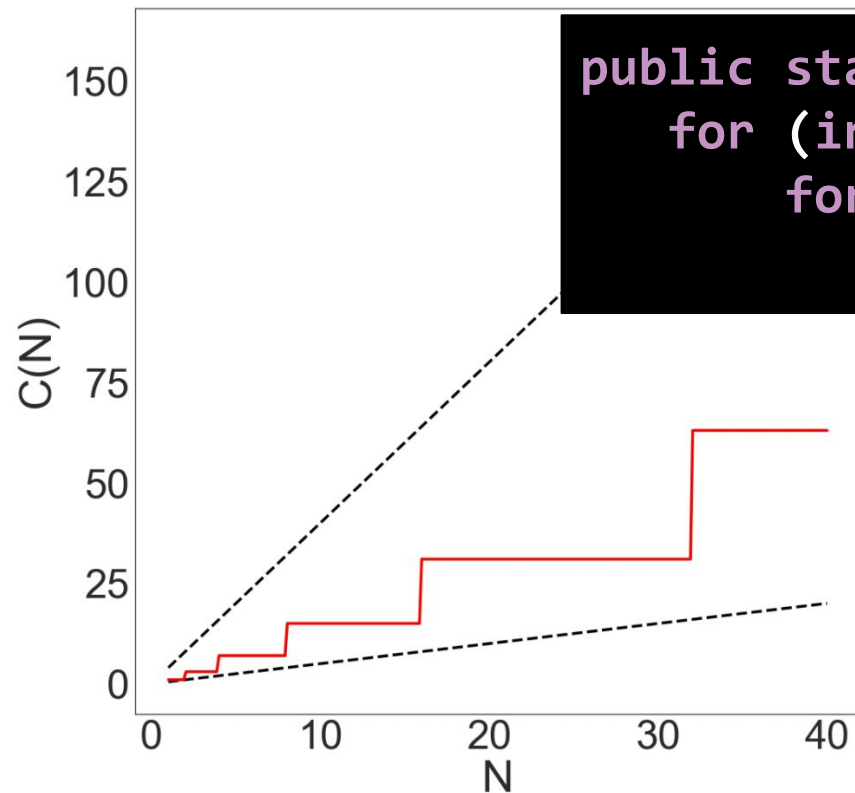
Solution: C(N) is Linear

Or looking at a table, we see that for large N, C(N) is always between 0.5N and 4N.

N	C(N)	0.5 N (below)	4N (above)
1	1	0.5	8
4	1 + 2 + 4 = 7	2	16
7	1 + 2 + 4 = 7	3.5	28
8	1 + 2 + 4 + 8 = 15	4	32
27	1 + 2 + 4 + 8 + 16 = 31	13.5	108
185	... + 64 + 128 = 255	92.5	740
715	... + 256 + 512 = 1023	357.5	2,860

We saw that printParty is $\Theta(N)$.

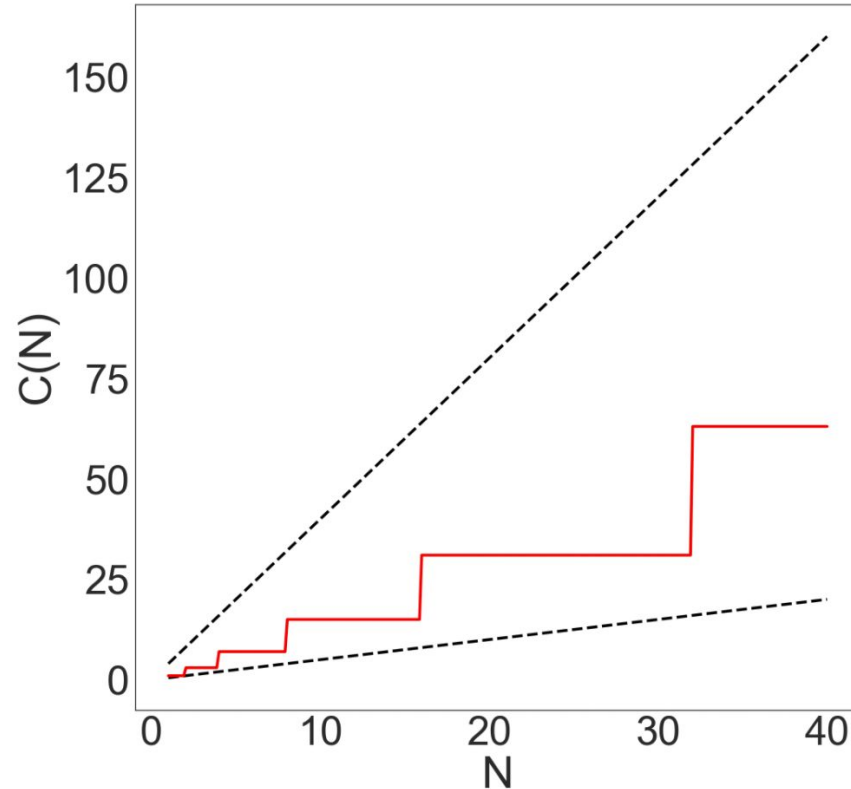
- What data structure we've discussed matches this pattern?



```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
        }  
    }  
}
```

We saw that printParty is $\Theta(N)$.

- What data structure we've discussed matches this pattern?



Your answer:

-

There is No Magic Shortcut for Asymptotic Analysis

Lecture 14, CS61B, Fall 2025

Asymptotic Analysis Examples

- Nested For Loops
- A Trickier Nested For Loop
- **There is No Magic Shortcut for Asymptotic Analysis**
- Recursive Function Analysis

More Asymptotic Notation

- Big O
- Big Theta

There is no magic shortcut for asymptotic analysis problems (well... [usually](#))

- Runtime analysis often requires careful thought.
- CS70 and especially CS170 will cover this in much more detail.
- This is not a math class, though we'll expect you to know these:
 - $1 + 2 + 3 + \dots + Q = Q(Q+1)/2 = \Theta(Q^2)$ ← Sum of First Natural Numbers ([Link](#))
 - $1 + 2 + 4 + 8 + \dots + Q = 2Q - 1 = \Theta(Q)$ ← Sum of First Powers of 2 ([Link](#))

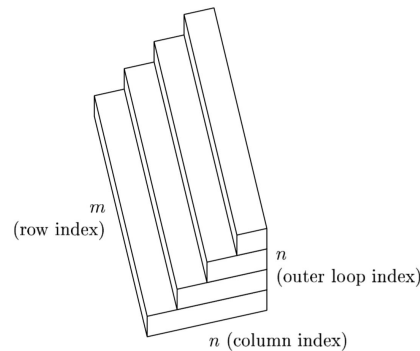
Where Q is a power of 2.

```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```

There is no magic shortcut for asymptotic analysis problems (well... [usually](#))

- Runtime analysis often requires careful thought.
- CS70 and especially CS170 will cover this in much more detail.
- This is not a math class, though we'll expect you to know these:
 - $1 + 2 + 3 + \dots + Q = Q(Q+1)/2 = \Theta(Q^2)$ ← Sum of First Natural Numbers ([Link](#))
 - $1 + 2 + 4 + 8 + \dots + Q = 2Q - 1 = \Theta(Q)$ ← Sum of First Powers of 2 ([Link](#))
- Strategies:
 - Find exact sum.
 - Write out examples.
 - Draw pictures.

QR decomposition runtime,
from “Numerical Linear
Algebra” by Trefethen.



The $m \times n$ rectangle at the bottom corresponds to the first pass through the outer loop, the $m \times (n - 1)$ rectangle above it to the second pass, and so on.

Big O

Lecture 14, CS61B, Fall 2025

Asymptotic Analysis Examples

- Nested For Loops
- A Trickier Nested For Loop
- There is No Magic Shortcut for Asymptotic Analysis
- Recursive Function Analysis

More Asymptotic Notation

- **Big O**
- Big Theta

You might have noticed in the Disjoint Sets lecture that Peyrin used $O(N)$ for some of the entries in this table:

Implementation	Constructor	connect	isConnected
ListOfSetsDS	$\Theta(N)$	$O(N)$	$O(N)$
QuickFindDS	$\Theta(N)$	$\Theta(N)$	$\Theta(1)$
QuickUnionDS	$\Theta(N)$	$O(N)$	$O(N)$
WeightedQuickUnionDS	$\Theta(N)$	$O(\log N)$	$O(\log N)$

Let's unpack what this means!

- First, let's review $\Theta(N)$.

We used Big Theta to describe the order of growth of a function.

function $R(N)$	order of growth
$N^3 + 3N^4$	$\Theta(N^4)$
$1/N + N^3$	$\Theta(N^3)$
$1/N + 5$	$\Theta(1)$
$N^N + N$	$\Theta(N^N)$
$40 \sin(N) + 4N^2$	$\Theta(N^2)$

We also used Big Theta to describe the rate of growth of the runtime of a piece of code.

- Idea was simple: Assume runtime is a function of the input.
- Example: Assume runtime of `printParty` is proportional to # of prints.

Whereas Big Theta can informally be thought of as something like “equals”, Big O can be thought of as “less than or equal”.

Example, the following are all true:

- $N^3 + 3N^4 \in \Theta(N^4)$
- $N^3 + 3N^4 \in O(N^4)$
- $N^3 + 3N^4 \in O(N^6)$
- $N^3 + 3N^4 \in O(N!)$
- $N^3 + 3N^4 \in O(N^{N!})$

$$R(N) \in \Theta(f(N))$$

means there exist positive constants k_1 and k_2 such that:

$$k_1 \cdot f(N) \leq R(N) \leq k_2 \cdot f(N)$$

for all values of N greater than some N_0 .

 i.e. very large N

Example: $40 \sin(N) + 4N^2 \in \Theta(N^2)$

- $R(N) = 40 \sin(N) + 4N^2$
- $f(N) = N^2$
- $k_1 = 3$
- $k_2 = 5$

$$R(N) \in O(f(N))$$

means there exists a positive constant k_2 such that:

$$R(N) \leq k_2 \cdot f(N)$$

for all values of N greater than some N_0 .

 i.e. very large N

Example: $40 \sin(N) + 4N^2 \in O(N^4)$

- $R(N) = 40 \sin(N) + 4N^2$
- $f(N) = N^4$
- $k_2 = 10$

Big Theta vs. Big O

	Informal meaning:	Family	Family Members
Big Theta $\Theta(f(N))$	Order of growth is $f(N)$.	$\Theta(N^2)$	$N^2/2$ $2N^2$ $N^2 + 38N + N$
Big O $O(f(N))$	Order of growth is less than or equal to $f(N)$.	$O(N^2)$	$N^2/2$ $2N^2$ $\lg(N)$

Why are the four entries (bottom right, in pink) given in O notation rather than Θ ?



Implementation	Constructor	connect	isConnected
QuickFindDS	$\Theta(N)$	$\Theta(N)$	$\Theta(1)$
QuickUnionDS	$\Theta(N)$	$O(N)$	$O(N)$
WeightedQuickUnionDS	$\Theta(N)$	$O(\log N)$	$O(\log N)$

Why are the four entries (bottom right, in pink) given in O notation rather than Θ ?

- Might be linear, might be constant.
- MORE ON THIS LATER.

Implementation	Constructor	connect	isConnected
QuickFindDS	$\Theta(N)$	$\Theta(N)$	$\Theta(1)$
QuickUnionDS	$\Theta(N)$	$O(N)$	$O(N)$
WeightedQuickUnionDS	$\Theta(N)$	$O(\log N)$	$O(\log N)$

Big Theta

Lecture 14, CS61B, Fall 2025

Asymptotic Analysis Examples

- Nested For Loops
- A Trickier Nested For Loop
- There is No Magic Shortcut for Asymptotic Analysis
- Recursive Function Analysis

More Asymptotic Notation

- Big O
- **Big Theta**

Whereas Big Theta can informally be thought of as something like “equals”, Big Omega can be thought of as “greater than or equal”.

Example, the following are all true:

- $N^3 + 3N^4 \in \Theta(N^4)$
- $N^3 + 3N^4 \in \Omega(N^4)$
- $N^3 + 3N^4 \in \Omega(N^3)$
- $N^3 + 3N^4 \in \Omega(\log N)$
- $N^3 + 3N^4 \in \Omega(1)$

$$R(N) \in \Omega(f(N))$$

means there exists a positive constant k_1 such that:

$$k_1 \cdot f(N) \leq R(N)$$

for all values of N greater than some N_0 .

 i.e. very large N

Example: $40 \sin(N) + 4N^2 \in \Omega(N)$

- $R(N) = 40 \sin(N) + 4N^2$
- $f(N) = N$
- $k_1 = 20$

Big Theta, Big O, and Big Omega

	Informal meaning:	Family	Family Members
Big Theta $\Theta(f(N))$	Order of growth is $f(N)$.	$\Theta(N^2)$	$N^2/2$ $2N^2$ $N^2 + 38N + N$
Big O $O(f(N))$	Order of growth is less than or equal to $f(N)$.	$O(N^2)$	$N^2/2$ $2N^2$ $\lg(N)$
Big Omega $\Omega(f(N))$	Order of growth is greater than or equal to $f(N)$.	$\Omega(N^2)$	$N^2/2$ $2N^2$ e^N

Big Omega is often used for setting theoretical floors on problem difficulty.

For example, finding the median of a list of numbers takes $\Omega(N)$.

- That is, among the space of ALL possible algorithms for finding the median, the best you could ever do is linear time. Why?

We'll use Ω in a much later lecture.

Big Theta, Big O, and Big Omega Summary (Again)

	Informal meaning:	Family	Family Members
Big Theta $\Theta(f(N))$	Order of growth is $f(N)$.	$\Theta(N^2)$	$N^2/2$ $2N^2$ $N^2 + 38N + N$
Big O $O(f(N))$	Order of growth is less than or equal to $f(N)$.	$O(N^2)$	$N^2/2$ $2N^2$ $\lg(N)$
Big Omega $\Omega(f(N))$	Order of growth is greater than or equal to $f(N)$.	$\Omega(N^2)$	$N^2/2$ $2N^2$ e^N

Next up:

- HW7 due today (extra time due to Midterm).
- HW8 due Friday.

